Proyecto: HORAPUNTA
Asignatura: IASI (Inteligencia Artificial y Sistemas Inteligentes)
Autores: Eduardo Cabello Gil, Carlos Paredes Ruiz y Gabriel Ramos Sánchez

PROYECTO DE LA ASIGNATURA DE INTELIGENCIA ARTIFICIAL Y SISTEMAS INTELIGENTES 2025/2026

HORA PUNTA

1. Descripción de la instalación, interfaz y manejo de la práctica

La forma de ejecución de este proyecto en la plataforma Eclipse es sencilla e intuitiva.

Primero debemos abrir Eclipse C/C++ e importamos el proyecto seleccionando “Proyecto existente”.

Una vez importado correctamente abriremos el archivo llamado *main.cpp* y una vez teniéndolo abierto lo vamos a compilar y ejecutar de forma *Local*.

Acto seguido en la parte inferior de la ventana de *Eclipse* se abrirá en la *terminal* una interfaz creada por nosotros para el proyecto.

Uso de interfaz:

Una vez lanzado el programa, la interfaz consiste en una *lectura por consola.*

Primero se te pedirá qué tablero quieres seleccionar para la ejecución de los algoritmos, acto seguido, se te vuelve a solicitar un número para elegir un algoritmo específico de búsqueda a ejecutar. La interfaz se verá de la siguiente manera:

…

Seleccione un tablero para jugar (1-10) o 0 para salir:
>>A continuación el usuario deberá marcar un número de su elección.
Se muestra el tablero correspondiente al número introducido por el usuario.
Elige qué algoritmo usar para intentar resolver el tablero elegido:
0 – Salir del programa.
1 – Escalada Simple.
2 – Escalada Máxima Pendiente.
3 – Escalada Simple Estocástica.
4 – Algoritmo A.*
>> A continuación el usuario deberá marcar un número de su elección.
Se ejecuta el algoritmo elegido sobre el tablero anterior.

…

El programa mostrará los pasos seguidos (esto es, los nodos extendidos) hasta encontrar un estado objetivo: que el vehículo **Z** se encuentre en cualquiera de los bordes del tablero.

Si no ha sido capaz de encontrar un estado objetivo junto con una solución para llegar a él, imprimirá un *mensaje por pantalla* avisando de tal hecho.

2. Métodos de desarrollo del software

Nuestro proyecto ha sido desarrollado plenamente por nosotros aunque hemos optado por un flujo de trabajo mixto entre IA y usuario.

Al inicio del proyecto, valoramos una opción muy útil y práctica: usar el IDE ***VSCode*** ya que nos permitía un acceso al chat de IA con el modelo de lenguaje preferido (Claude, Gemini o GPT). Para ello hemos usado la técnica de prompting ***"Socratic prompting"*** en donde la IA no te ofrece directamente el código ni la solución correcta sino que te hace preguntas guiadas, te ayuda a razonar paso a paso y detecta fallos en tu lógica.

Estos **prompts** junto con otra técnica de *prompting denominada* ***"Coach prompting"*** nos han acompañado en el desarrollo de este proyecto, específicamente con la implementación de algoritmos de búsqueda.

Retomando el motivo de uso del IDE ***VSCode*** *queremos dejar claros los principales motivos que nos han llevado a usar este entorno en lugar del recomendado* **Eclipse IDE***:*

-***Integración con GitHub****:* al ser un proyecto desarrollado por 3 integrantes la forma más eficiente era crear un repositorio en donde cada uno pueda actualizar los archivos del proyecto de una forma ordenada, además de tener un historial de todos los cambios y versiones que cada uno ha implementado.

-**Uso de la extensión *LiveShare***: esta extensión nos ha permitido editar el mismo código desde distintas máquinas al mismo tiempo de una forma similar a como funcionan los documentos de ***Google***. Esto nos ha permitido un flujo de trabajo mejor orientado, donde cada integrante puede ver cambios en tiempo real que realizan los compañeros a la vez que puede arreglar código e implementar nuevos métodos.

3. Descripción de los algoritmos de resolución usados en la solución

El principal archivo que implementa todos estos algoritmos se llama **MetodosBusqueda.*cpp* .**

En este archivo se detallan las estrategias de búsqueda y los algoritmos implementados para resolver los tableros de ***HORAPUNTA.*** La solución se basa en técnicas de búsqueda heurística, variando desde métodos de mejora local hasta un algoritmo de búsqueda global óptima.

3.1. Estructuras de Datos Principales

Para la gestión eficiente del estado del juego y el proceso de búsqueda, se han utilizado las siguientes estructuras:

- ***Tablero:*** Representa el estado actual del juego mediante una matriz de caracteres ***(vector<vector<char>>)***. Gestiona la disposición de los vehículos y el hueco vacío (***X***).

- ***Nodo:*** Usada únicamente en el algoritmo A*. Almacena el estado del tablero, un puntero al nodo padre (para reconstruir la solución), el coste acumulado (***g***), el valor de la heurística (***h***) y el carácter que representa el vehículo movido para llegar a este estado.
- ***Vector de pares (vector<pair<int,int>>):*** Se utiliza para delimitar la extensión de los vehículos en el tablero durante el proceso que suponen sus movimientos en él.

3.2. Función Heurística

La **heurística** diseñada estima la distancia al objetivo evaluando la facilidad con la que el vehículo ***Z*** puede alcanzar un borde del tablero teniendo en cuenta 4 factores: Distancia de Z al borde, número de vehículos bloqueadores, coste de movilidad de X y una bonificación si X está en una esquina del tablero.

Los tres primeros factores se evalúan para los cuatro sentidos en los que se puede mover el vehículo Z (arriba, abajo, izquierda y derecha) con el objetivo de quedarnos con el valor de heurística más pequeño de entre las cuatro opciones. Hemos decidido implementar la heurística de este modo ya que nos ha parecido lógico pensar que deberíamos darle un valor heurístico más favorable a un posible movimiento del vehículo Z en una dirección que: esté a menos casillas de distancia de un borde, tenga menos vehículos bloqueando ese camino y cueste menos llevar la X a ese lado de Z para conseguir moverlo.

A continuación se presenta una descripción un poco más específica de cada uno de los factores:

1. Distancia al borde: Cuántas casillas tiene que recorrer Z para llegar al borde correspondiente a ese sentido.

2. Vehículos Bloqueadores: se utiliza la función contarBloqueadoresEnLinea, que identifica cuántos vehículos obstruyen el camino de Z hacia el borde correspondiente a ese sentido. Este número de vehículos obstaculizadores se multiplica por 2 para dar más peso al hecho de que desbloquear un vehículo es costoso.

3. Coste de movilidad de X: se calcula la distancia Manhattan entre el hueco X y la posición necesaria para mover Z hacia ese sentido.

4. Bonificación por Esquina: si el hueco X está en una esquina del tablero, se aplica un ajuste negativo (bonificación) debido a la mayor versatilidad de movimiento que ofrece esta situación.

3.3 Algoritmos de Búsqueda Local (*Escalada*)

Se han implementado tres variantes del algoritmo de escalada (***Hill Climbing***):

- **Escalada Simple:** Desde el estado actual se evalúan uno a uno los sucesores y se selecciona el primero que mejore la heurística actual.

- **Escalada por Máxima Pendiente:** Desde el estado actual se evalúan todos los posibles movimientos y se selecciona aquel que ofrece la mayor reducción en el valor de la heurística de entre ellos.
- **Escalada Simple Estocástica:** variante de la ***simple*** que introduce el umbral (***α***) para decidir si se acepta o no un movimiento, permitiendo una exploración ligeramente distinta.

Algoritmo General:

```
Procedimiento EscaladaSimple(tableroInicial)
    tableroActual = tableroInicial
    Mientras no esObjetivo(tableroActual) y iter < iterMax hacer:
        heuristicaActual = calcularHeuristica(tableroActual)
        encontradoMejor = falso
        Para cada movimiento en {arriba, abajo, izquierda, derecha} hacer:
            nuevoTablero = moverVehiculo(tableroActual, movimiento)
            Si heuristica(nuevoTablero) < heuristicaActual entonces:
                tableroActual = nuevoTablero
                encontradoMejor = verdadero
                Salir del bucle Para
        Si no encontradoMejor entonces:
            Terminar (Mínimo Local)
Fin Procedimiento
```

Imágen creada con CodeSnap

3.4. Algoritmo de Búsqueda Global (*A**)

El algoritmo A* evalúa un estado teniendo en cuenta dos criterios a la vez: que esté cerca posiblemente de encontrar en pocas operaciones o movimientos el estado final (función h') Y que el coste de llegar desde el estado inicial hasta este estado sea bajo (función g).

Mediante la suma de ambos criterios se forma la función f', que será la que se utilice para valorar si un estado es más cercano al objetivo que otro.

Técnicas de Variación empleadas:

- **Gestión de duplicados:** si al generar un nodo se observa que ya existe en la lista de nodos ***abiertos*** o ***cerrados*** pero con un valor de f' mayor, el algoritmo actualiza su coste y su puntero al nodo padre, pasando a apuntar al del nodo que se acaba de generar.
- **Programación de mejoras:** mediante ***actualizarDescendientesRecursivo***, si un nodo se encuentra ya en ***cerrados***, se actualizan recursivamente todos sus descendientes para mantener la consistencia del coste.

Pseudocódigo *A*:*

```
Procedimiento AlgoritmoAEstrella(tableroInicial)
    Crear nodoRaiz con g=0, h=calcularHeuristica(tableroInicial)
    Añadir nodoRaiz a lista ABIERTOS
    Mientras ABIERTOS no esté vacía hacer:
        nodoActual = Extraer nodo con menor f' de ABIERTOS (f' = g + h)
        Si esObjetivo(nodoActual.estado) entonces:
            Retornar reconstruirSolucion(nodoActual)

        Añadir nodoActual a CERRADOS
        Para cada movimiento posible hacer:
            nuevoEstado = aplicarMovimiento(nodoActual.estado)
            nodoHijo = CrearNodo(nuevoEstado, g = nodoActual.g + 1)

            Si nodoHijo no está en ABIERTOS ni en CERRADOS:
                Añadir nodoHijo a ABIERTOS
            Sino si f'_nueva < f'_antigua:
                Actualizar nodo existente en la lista correspondiente
                Si estaba en CERRADOS, actualizar descendientes recursivamente
        Ordenar ABIERTOS por f'
Fin Procedimiento
```

Imágen creada con CodeSnap

3.5. Lógica de movimiento (*moverVehiculo*)

Este algoritmo, que permite mover los vehículos hacia el hueco X, constituye nuestro **operador de transformación**. Se trata de una parte fundamental de la resolución de los tableros ya que ese el responsable de transformar un estado en otro.

El algoritmo identifica el hueco ***X***, determina qué vehículo adyacente se pretende mover según la dirección (***up, down, left , right***), y utiliza la función recursiva ***delimitarVehiculo*** para calcular el tamaño del vehículo y verificar si su desplazamiento es físicamente posible en función de la orientación del vehículo respecto a X y de si se encuentra dentro de los limites del tablero de *6 x 6.*

4. Resultados obtenidos en las baterías de pruebas para cada una de las soluciones

TABLERO	TÉCNICA DE RESOLUCIÓN	SOLUCIÓN	TIEMPO DE EJECUCIÓN*	NODOS GENERADOS
HORAPUNTA1	Escalada simple	D, C, Z	320 µs	7
HORAPUNTA1	Escalada Máxima Pendiente	D, C, Z	366 µs	12
HORAPUNTA1	Escalada Simple Estocástica	No se ha hallado la solución.	249 µs	8
HORAPUNTA1	Algoritmo A*	D, C, Z	826 µs	12
HORAPUNTA2	Escalada simple	C, A, Z	365 µs	8
HORAPUNTA2	Escalada Máxima Pendiente	C, A, Z	376 µs	12
HORAPUNTA2	Escalada Simple Estocástica	No se ha hallado la solución	110 µs	4
HORAPUNTA2	Algoritmo A*	C, A, Z	713 µs	12
HORAPUNTA3	Escalada simple	B	176 µs	5
HORAPUNTA3	Escalada Máxima Pendiente	B	216 µs	8

HORAPUNTA3	Escalada Simple Estocástica	No se ha hallado la solución.	109 µs	4
HORAPUNTA3	Algoritmo A*	B, A, J, S, G, I, Z	2386 µs	40
HORAPUNTA4	Escalada simple	No se ha hallado la solución.	294 µs	6
HORAPUNTA4	Escalada Máxima Pendiente	No se ha hallado la solución.	243 µs	8
HORAPUNTA4	Escalada Simple Estocástica	No se ha hallado la solución.	115 µs	5
HORAPUNTA4	Algoritmo A*	No se ha hallado la solución.	587 µs	16
HORAPUNTA5	Escalada simple	E, T, J, A, Z	444 µs	13
HORAPUNTA5	Escalada Máxima Pendiente	E, T, J, A, Z	694 µs	20
HORAPUNTA5	Escalada Simple Estocástica	No se ha hallado la solución.	245 µs	5
HORAPUNTA5	Algoritmo A*	E, T, J, A, Z	1442 µs	20
HORAPUNTA6	Escalada simple	No se ha hallado la solución.	334 µs	8
HORAPUNTA6	Escalada Máxima Pendiente	No se ha hallado la solución.	301 µs	8
HORAPUNTA6	Escalada Simple Estocástica	No se ha hallado la solución.	249 µs	6
HORAPUNTA6	Algoritmo A*	No se ha hallado la solución.	8094 µs	88
HORAPUNTA7	Escalada simple	No se ha hallado la solución.	247 µs	7
HORAPUNTA7	Escalada Máxima Pendiente	No se ha hallado la solución.	369 µs	8
HORAPUNTA7	Escalada Simple Estocástica	No se ha hallado la solución	245 µs	7
HORAPUNTA7	Algoritmo A*	No se ha hallado la solución.	599 µs	16
HORAPUNTA8	Escalada simple	No se ha hallado la solución.	293 µs	7
HORAPUNTA8	Escalada Máxima Pendiente	No se ha hallado la solución.	391 µs	8
HORAPUNTA8	Escalada Simple Estocástica	No se ha hallado la solución.	171 µs	4
HORAPUNTA8	Algoritmo A*	J, L, Z, J, L, N, M, Z	7926 µs	80
HORAPUNTA9	Escalada simple	El tablero ya está resuelto, no es necesario realizar ningún movimiento.	32 µs	0
HORAPUNTA9	Escalada Máxima Pendiente	El tablero ya está resuelto, no es necesario realizar ningún movimiento.	32 µs	0
HORAPUNTA9	Escalada Simple Estocástica	El tablero ya está resuelto, no es necesario realizar ningún movimiento.	36 µs	0
HORAPUNTA9	Algoritmo A*	El tablero ya está resuelto, no es necesario realizar ningún movimiento.	112 µs	0
HORAPUNTA10	Escalada simple	No se ha hallado la solución.	205 µs	4
HORAPUNTA10	Escalada Máxima Pendiente	No se ha hallado la solución.	206 µs	4
HORAPUNTA10	Escalada Simple Estocástica	No se ha hallado la solución.	229 µs	4
HORAPUNTA10	Algoritmo A*	No se ha hallado la solución.	18881 µs	120

*Cabe destacar que los tiempos de ejecución serán distintos en función de la máquina en la que se ejecute el proyecto.

5. Conclusiones sobre los resultados para cada una de las técnicas de resolución empleadas.

Eficacia y Tasa de Acierto

- **Algoritmo A*:** ha demostrado ser el método más robusto, siendo el único capaz de resolver tableros complejos como ***HORAPUNTA8***. Su tasa de acierto es la más alta debido a que explora todo el árbol de estados siempre que la memoria lo permita.
- **Escalada Simple y Máxima Pendiente:** son extremadamente rápidas en tableros sencillos (***HORAPUNTA1, HORAPUNTA 2*** **u** ***HORAPUNTA5***), pero presentan una alta tendencia a quedar atrapadas en **mínimos locales** en tableros con muchos bloqueadores, como se observa en ***HORAPUNTA4*** u ***HORAPUNTA6***, donde no hallaron solución. En definitiva, son algoritmos incapaces de tomar un camino con una heurística peor pero que posteriormente lleve a la solución.

- **Escalada Simple Estocástica:** ha demostrado con diferencia el peor rendimiento en este escenario específico. La introducción del valor umbral α parece dificultar la convergencia en un juego de puzzle tan restrictivo como este, donde los movimientos válidos son limitados.

Eficiencia Temporal y Espacial

- **Nodos Generados:** Existe una correlación directa entre la complejidad del tablero y el consumo de memoria en *A**. Por ejemplo, en el tablero ***HORAPUNTA10***, ***A**** llegó a generar **120** nodos antes de determinar la falta de solución, con un tiempo de **18.881** μs .
- **Tiempos de Ejecución**: Los algoritmos de Escalada son órdenes de magnitud más rápidos (una **media de 200-400 μs**) comparados con *A** cuando este último debe profundizar en el árbol de búsqueda.

Recomendación de Uso

- Se recomienda el uso de ***A**** para ***garantizar la resolución*** del puzle en cualquier configuración, a pesar del coste computacional.
- Las técnicas de Escalada son **adecuadas** únicamente como filtros rápidos iniciales o para configuraciones de tablero ***con baja densidad de vehículos*** (pocos bloqueadores) .

8. Referencias.

Para el desarrollo de este proyecto, se han consultado y utilizado los siguientes recursos:

I. Documentación y Entornos

- **Eclipse Foundation**: *IDE Eclipse 4.36 (C/C++ Development Tooling)* sobre Ubuntu.
- **Microsoft**: *Visual Studio Code* y extensión *LiveShare* para el desarrollo colaborativo en tiempo real.
- **GitHub**: Plataforma para el control de versiones y gestión del repositorio del equipo.

II. Inteligencia Artificial y Herramientas de Apoyo

- **Modelos de Lenguaje** (*Claude/Gemini/GPT*): utilizados mediante técnicas de Socratic Prompting y Coach Prompting para el razonamiento de algoritmos de búsqueda y detección de errores lógicos.
- **CodeSnap**: Herramienta utilizada para la generación de las imágenes de pseudocódigo incluidas en esta documentación.

III.Bibliografía Técnica

- Material docente de la asignatura Inteligencia Artificial y Sistemas Inteligentes 2025/2026 (especificaciones de la práctica y heurísticas de búsqueda).

- Documentación estándar de las siguientes librerías de C++: ***iostream***, para gestionar las operaciones de entrada y salida de datos (cin/cout); ***vector***, la librería que más hemos utilizado con diferencia, principalmente para almacenar en memoria los tableros en forma de matrices y poder trabajar sobre ellos; ***algorithm***, útil para ordenar el vector de "abiertos" en A*; ***map***, para almacenar en la misma estructura de datos las coordenadas que ocupan los vehículos en el tablero al moverlos; ***fstream***, utilizada para leer los tableros de los ficheros de texto; ***set***, empleado para almacenar los vehículos bloqueadores desde Z hasta el borde para evitar duplicados; ***chrono***, para calcular los tiempos de ejecución de los algoritmos; ***random***, utilizado para aleatorizar el orden en que se generan los nodos en la escalada simple estocástica; y ***filesystem***, que permite incluir nuevos tableros al proyecto y que se puedan usar sin tener que realizar cambios mayores en el código.